

Part 4: Iterative Traversals

Replacing the call stack with your own stack

Why bother

Recursion already works and is shorter. So why learn the iterative versions?

1. **Interviewers ask for it.** “Now do it without recursion” is a standard follow-up.
2. **Deep trees overflow the call stack.** A skewed tree with 10^5 nodes means 10^5 pending calls. Your own stack lives on the heap and does not have that limit.
3. **You get control.** Pausing a traversal, resuming it later, or implementing a BST iterator (LeetCode 173) is easy with an explicit stack and impossible with plain recursion.
4. **It teaches you what recursion actually does.** After this part, Morris traversal in Part 12 will look like a natural next step instead of black magic.

What the call stack actually stores

When `preorder(2)` calls `preorder(4)`, the machine has to remember two things about the paused call:

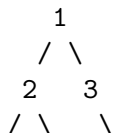
- **which node** it was working on (node 2)
- **where to resume** (it already finished the left call, so next is the right call)

That is the entire content of a stack frame for our purposes. An iterative traversal keeps exactly the same information by hand, in a `stack<TreeNode*>` plus the structure of the loop.

```
stack<TreeNode*> st;
st.push(node);    // remember to come back to this node
st.top();         // the most recently postponed node
st.pop();         // done with it
```

A stack is last in, first out. That matches recursion: the most recently postponed call is the one you resume first.

Throughout this part, the tree is:



4 5 6

with preorder 1 2 4 5 3 6, inorder 4 2 5 1 3 6, postorder 4 5 2 6 3 1.

Iterative preorder

This is the easy one, because preorder records a node the moment it arrives. There is nothing to postpone about the node itself, only about its children.

Algorithm: push the root. Repeatedly pop a node, record it, then push its right child and then its left child.

```
vector<int> preorderTraversal(TreeNode* root) {
    vector<int> out;
    if (root == NULL) return out;

    stack<TreeNode*> st;
    st.push(root);

    while (!st.empty()) {
        TreeNode* node = st.top();
        st.pop();

        out.push_back(node->val);

        if (node->right) st.push(node->right);    // right first
        if (node->left) st.push(node->left);      // left on top, so it pops first
    }
    return out;
}
```

Why right is pushed before left

A stack reverses order. We want left to be processed first, so left must be on top, so left goes in last. Swap the two lines and you get the “root, right, left” order, which is a genuinely useful traversal in its own right, as you will see in the postorder section.

Dry run

push 1	st = [1]	out = []
pop 1, record 1, push 3, 2	st = [3,2]	out = [1]
pop 2, record 2, push 5, 4	st = [3,5,4]	out = [1,2]
pop 4, record 4, no kids	st = [3,5]	out = [1,2,4]
pop 5, record 5, no kids	st = [3]	out = [1,2,4,5]
pop 3, record 3, push 6	st = [6]	out = [1,2,4,5,3]
pop 6, record 6	st = []	out = [1,2,4,5,3,6]

Matches the recursive output.

Time $O(N)$. Space $O(H)$ for the stack, which is the same as the recursion it replaces.

Recursive versus iterative

	Recursive	Iterative
Lines of code	4	12
Stack used	call stack	your own stack<TreeNode*>
Deep tree risk	can overflow	safe
Readability	better	acceptable

For preorder there is no reason to prefer the iterative version except when the tree can be extremely deep, or when the interviewer asks.

Iterative inorder

Now it gets interesting. Inorder cannot record a node when it arrives, because the entire left subtree has to come out first. So arriving at a node means “postpone it and dive left”.

Algorithm:

```
curr = root
repeat:
    while curr is not empty:
        push curr
        curr = curr.left           <- dive as far left as possible
    if stack is empty: stop
    pop node, record node         <- left subtree is finished, so this node is next
    curr = node.right             <- now handle the right subtree the same way
```

```
vector<int> inorderTraversal(TreeNode* root) {
    vector<int> out;
    stack<TreeNode*> st;
    TreeNode* curr = root;

    while (curr != NULL || !st.empty()) {
        while (curr != NULL) {           // go left as far as possible
            st.push(curr);
            curr = curr->left;
        }
        curr = st.top();                  // leftmost unfinished node
        st.pop();
        out.push_back(curr->val);
        curr = curr->right;               // switch to the right subtree
    }
    return out;
}
```

The loop condition

while (curr != NULL || !st.empty()) looks odd until you name the two situations:

- curr != NULL means “there is a subtree I have not started yet”
- !st.empty() means “there are nodes I postponed and still owe an answer to”

The traversal is finished only when both are false. Writing only `!st.empty()` breaks the moment the stack empties while a right subtree is still pending, which happens at node 1 in our tree.

Dry run

```
curr=1, st=[]
  dive left: push 1, curr=2; push 2, curr=4; push 4, curr=NULL
  st=[1,2,4]
  pop 4, record 4, curr = 4.right = NULL      out=[4]      st=[1,2]
  no dive (curr NULL)
  pop 2, record 2, curr = 2.right = 5          out=[4,2]     st=[1]
  dive left from 5: push 5, curr = 5.left = NULL  st=[1,5]
  pop 5, record 5, curr = NULL                 out=[4,2,5]    st=[1]
  pop 1, record 1, curr = 1.right = 3           out=[4,2,5,1]  st=[]
  dive left from 3: push 3, curr = 3.left = NULL  st=[3]
  pop 3, record 3, curr = 3.right = 6           out=[4,2,5,1,3] st=[]
  dive: push 6, curr = NULL                     st=[6]
  pop 6, record 6, curr = NULL                   out=[4,2,5,1,3,6] st=[]
curr NULL and stack empty, stop
```

Time $O(N)$: each node is pushed once and popped once. **Space $O(H)$.**

Where this template gets reused

The BST iterator (LeetCode 173) is literally this loop, cut in half. The “dive left” part becomes a helper, and each call to `next()` performs one pop plus one dive. Kth smallest in a BST (LeetCode 230) is the same loop with a counter and an early return, which is strictly better than building the whole sorted list.

Iterative postorder

Postorder is the hardest, because a node may only be recorded after **both** subtrees are done, so when you pop a node from the stack you cannot immediately tell whether you are coming back from the left subtree or the right one.

There are three standard ways to handle this. Learn the first one properly, and know that the third exists.

Method 1, the reversal trick (learn this one)

Postorder is left, right, root. Reverse that sequence and you get root, right, left. And root, right, left is just preorder with the two pushes swapped, which we already know how to write.

So: do a preorder with right and left swapped, then reverse the result.

```
vector<int> postorderTraversal(TreeNode* root) {
    vector<int> out;
    if (root == NULL) return out;
```

```

stack<TreeNode*> st;
st.push(root);

while (!st.empty()) {
    TreeNode* node = st.top();
    st.pop();

    out.push_back(node->val);

    if (node->left) st.push(node->left);    // note: left pushed first now
    if (node->right) st.push(node->right);  // so right pops first
}

reverse(out.begin(), out.end());
return out;
}

```

Dry run

push 1	st=[1]	out=[]
pop 1, record 1, push 2, 3	st=[2,3]	out=[1]
pop 3, record 3, push 6	st=[2,6]	out=[1,3]
pop 6, record 6	st=[2]	out=[1,3,6]
pop 2, record 2, push 4, 5	st=[4,5]	out=[1,3,6,2]
pop 5, record 5	st=[4]	out=[1,3,6,2,5]
pop 4, record 4	st=[]	out=[1,3,6,2,5,4]

reverse -> [4,5,2,6,3,1]

Which is exactly the postorder.

Time $O(N)$, space $O(H)$ plus the $O(N)$ output which we were building anyway. The reversal is $O(N)$.

Downside: you do not get the nodes in postorder **as you go**, only at the end. That is fine for a traversal question and not fine if you wanted to free memory or compute values on the fly. In practice, when you actually need bottom-up computation, you use recursion, so this is rarely a problem.

Method 2, two stacks

The same idea written more explicitly. Push into a second stack instead of reversing at the end.

```

vector<int> postorderTwoStacks(TreeNode* root) {
    vector<int> out;
    if (root == NULL) return out;

    stack<TreeNode*> s1, s2;
    s1.push(root);

    while (!s1.empty()) {
        TreeNode* node = s1.top(); s1.pop();
        s2.push(node);
        if (node->left) s1.push(node->left);
        if (node->right) s1.push(node->right);
    }
}

```

```

    }
    while (!s2.empty()) {
        out.push_back(s2.top()->val);
        s2.pop();
    }
    return out;
}

```

Same complexity, one extra stack. Some people find it clearer. It is the same algorithm as method 1.

Method 3, one stack with a lastVisited pointer

This one produces true postorder in real time, but it needs one extra idea: remember which node you most recently finished, so that when you come back up to a node you can tell whether its right subtree is already done.

```

vector<int> postorderOneStack(TreeNode* root) {
    vector<int> out;
    stack<TreeNode*> st;
    TreeNode* curr = root;
    TreeNode* lastVisited = NULL;

    while (curr != NULL || !st.empty()) {
        while (curr != NULL) {           // dive left, same as inorder
            st.push(curr);
            curr = curr->left;
        }
        TreeNode* node = st.top();

        if (node->right != NULL && lastVisited != node->right) {
            curr = node->right;           // right subtree not done yet, go there
        } else {
            out.push_back(node->val);      // both subtrees done, record now
            lastVisited = node;
            st.pop();
        }
    }
    return out;
}

```

The condition `lastVisited != node->right` is the whole trick. It answers the question “have I just come back **from** my own right child?” If yes, the node is finished. If no, the right subtree still has to run.

Time $O(N)$, space $O(H)$.

Use method 1 in a contest. Use method 3 if the interviewer specifically asks for postorder without reversing, or if you must process nodes in true postorder while traversing.

Summary table

Traversal	Iterative idea	Extra state	Difficulty
Preorder	pop, record, push right then left	none	easy
Inorder	dive left pushing, pop, record, go right	none	medium
Postorder (method 1)	root right left, then reverse	none	easy
Postorder (method 3)	dive left, check if right is done	lastVisited	hard

All of them are $O(N)$ time and $O(H)$ space, exactly like the recursive versions. The iterative form does not make anything faster. It only moves the memory from the call stack to the heap and gives you control.

Common mistakes

- **Pushing left before right in iterative preorder.** You get root, right, left. Correct for the postorder trick, wrong for preorder.
- **Using `while (!st.empty())` alone in iterative inorder.** The stack is empty at the start and can be empty in the middle while work remains. You need the `curr != NULL` half too.
- **Pushing NULL onto the stack.** Guard every push with `if (node->left)`.
- **Reading `st.top()` after `st.pop()`.** Read first, pop second.
- **Trying to write postorder as “pop, record, push left, push right, no reversal”.** That is not postorder in any order. Either reverse, or use `lastVisited`.
- **Forgetting the final reverse in method 1.** You end up with root, right, left and cannot see why the answer looks “backwards”.
- **Believing the iterative version is asymptotically better.** It is not. The gain is stack safety and control.

Check yourself

1. What two pieces of information does a call stack frame hold in a traversal, and where do they live in the iterative version?
2. In iterative preorder, why must the right child be pushed before the left child?
3. Why can inorder not record a node when it first reaches it?
4. Explain the loop condition `curr != NULL || !st.empty()` in one sentence.
5. Why does reversing a “root, right, left” traversal produce postorder?
6. In method 3, what question is `lastVisited != node->right` really answering?
7. Is iterative traversal faster than recursive traversal? Justify your answer with complexities.